

🧩 FLEXBOX WORKSHEET – “From Zero to Flex Hero!”

💡 *Practice Sheet designed for students to learn by doing — concept → mini challenge → solution check.*

🧱 LEVEL 1: Warm-Up (Enable Flexbox)

🎯 **Goal:** Turn a parent <div> into a Flexbox container.

💻 **Task:**

Make the container use Flexbox.

```
<div class="container">

  <div class="box">1</div>

  <div class="box">2</div>

  <div class="box">3</div>

</div>

.container {

  /* 👉 Add the property to enable Flexbox */

}

.box {

  width: 60px;

  height: 60px;

  background-color: tomato;

  color: white;

  text-align: center;

  line-height: 60px;

  margin: 5px;

}
```

✅ **Hint:** Use display: flex;

💬 *Once you add it, boxes will align in a row automatically!*

🌀 LEVEL 2: Direction Master

🎯 **Goal:** Change layout direction (row → column → reverse order).

💻 **Try:**

```
.container {
```

```
display: flex;
flex-direction: _____; /* fill row / column / row-reverse / column-reverse */
}
```

Practice:

Task Expected Output

row  1 2 3

row-reverse  3 2 1

column  1
2
3

column-reverse  3
2
1

 **Think:** What changes when you rotate the axis?

LEVEL 3: Horizontal Alignment (justify-content)

 **Goal:** Position boxes horizontally in different ways.

 Try each value one by one:

```
.container {
display: flex;

justify-content: _____; /* Try flex-start, center, flex-end, space-between, space-around, space-evenly */
}
```

Value	Visual Meaning	Hint
flex-start	 (left side)	Default
center	 (centered)	Middle alignment
flex-end	 (right side)	Right side
space-between		No gaps at edges
space-around		Equal outer space
space-evenly		Equal space everywhere

LEVEL 4: Vertical Alignment (align-items)

 **Goal:** Align items vertically (cross-axis).

 **Try this:**

```
.container {  
  display: flex;  
  align-items: _____; /* Try flex-start, center, flex-end, stretch, baseline */  
  height: 200px;  
}
```

Value	Effect
-------	--------

flex-start	Items stick to top
------------	--------------------

center	Items vertically centered
--------	---------------------------

flex-end	Items stick to bottom
----------	-----------------------

stretch	Items fill full height
---------	------------------------

baseline	Align text baselines
----------	----------------------

 **Think:** The *cross axis* is **vertical** when direction = row.

LEVEL 5: Combo Power (Center Everything)

 **Goal:** Center all boxes both horizontally & vertically.

 **Challenge Code:**

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 250px;  
  background-color: lightgray;  
}
```

 **Output:** All boxes appear perfectly centered — this is one of the most common real-world uses of Flexbox!

 *Used in: loaders, modals, login forms, hero banners.*

LEVEL 6: Column Alignment Challenge

 **Goal:** Create vertically stacked boxes with space between.

 **Try:**

```
.container {  
  display: flex;  
  flex-direction: column;  
  justify-content: space-around;  
  align-items: center;  
  height: 300px;  
}
```

 **Output:**



Box 1

(space)

Box 2

(space)

Box 3

LEVEL 7: Mini LeetCode Challenge

 **Problem Statement:**

Arrange 4 boxes inside a container such that:

- Direction → row
- First box on left, last box on right
- Equal space between all boxes
- Vertically centered

 **Write the CSS:**

```
.container {  
  display: flex;  
  flex-direction: _____;  
  justify-content: _____;  
  align-items: _____;  
}
```

✅ **Expected Output:**

📦 — 📦 — 📦 — 📦 (centered vertically)

✖ **Answer Key:**

flex-direction: row; justify-content: space-between; align-items: center;

🌐 **LEVEL 8: Real-World Simulation**

🎯 **Task:** Create a *Website Navigation Bar* using Flexbox.

💻 **HTML**

```
<nav class="navbar">
  <div class="logo">MySite</div>
  <ul class="menu">
    <li>Home</li>
    <li>About</li>
    <li>Courses</li>
    <li>Contact</li>
  </ul>
</nav>
```

💻 **CSS**

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #2196f3;
  color: white;
  padding: 10px 20px;
}

.menu {
  display: flex;
  gap: 20px; /* space between items */
  list-style: none;
}
```

✅ **Output:** Logo on left, menu items on right — perfectly aligned!

🧠 LEVEL 9: Debug Challenge

🐛 A student wrote:

```
.container {  
  display: flex;  
  justify-content: bottom;  
}
```

❓ **Q:** Why isn't it working?

💡 **A:** There is no value bottom in Flexbox. Use align-items: flex-end; for vertical bottom alignment.

🎯 FINAL LEVEL: “Flexbox Boss Challenge”

👉 **Goal:** Create a layout that looks like this:

```
| Header      |  
|-----|  
| Sidebar | Main |  
| Footer   |
```

💡 **Hint:**

- Use flex-direction: column for full layout.
- Inside middle section, make another flex container for sidebar + main.

💡 *Nested flexboxes = pro-level mastery!*

📄 Summary – Flexbox Formula Sheet

Property	Axis	Values	Meaning
display: flex	—	—	Enables Flexbox
flex-direction	Main Axis	row / column	Controls direction
justify-content	Main Axis	start / center / end / space-*	Horizontal alignment
align-items	Cross Axis	start / center / end / stretch	Vertical alignment

LEVEL 1: Warm-Up (Enable Flexbox)

 **Goal:** Turn parent <div> into Flexbox container.

 **Solution:**

```
.container {  
  display: flex;  
}
```

 Once added → all .box items align in a row automatically (default direction = row).

LEVEL 2: Direction Master

 **Goal:** Change layout direction.

```
.container {  
  display: flex;  
  flex-direction: row;      /* default → 1 2 3 */  
  flex-direction: row-reverse; /* reverse → 3 2 1 */  
  flex-direction: column;   /* vertical → 1,2,3 */  
  flex-direction: column-reverse; /* reverse vertical → 3,2,1 */  
}
```

 **Concept:**

- flex-direction changes **main axis** (horizontal or vertical).
- Row = left to right
- Column = top to bottom

LEVEL 3: Horizontal Alignment (justify-content)

 **Goal:** Position boxes horizontally.

```
.container {  
  display: flex;  
  justify-content: flex-start; /* left side (default) */  
  justify-content: center;    /* centered horizontally */  
  justify-content: flex-end;  /* right side */  
  justify-content: space-between; /* no space on edges */  
  justify-content: space-around; /* equal outer spacing */  
  justify-content: space-evenly; /* equal space all around */  
}
```

 **Main axis = Horizontal line** when direction = row.

LEVEL 4: Vertical Alignment (align-items)

 **Goal:** Align items vertically (cross-axis).

```
.container {
```



```
display: flex;
align-items: flex-start; /* top */
align-items: center;    /* middle vertically */
align-items: flex-end;  /* bottom */
align-items: stretch;  /* fills height */
align-items: baseline;  /* align text baseline */
height: 200px;
}
```

💡 **Cross-axis** is **vertical** when direction = row.

🌟 **LEVEL 5: Combo Power (Center Everything)**

🎯 **Goal:** Center items both horizontally & vertically.

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 250px;
  background-color: lightgray;
}
```

✅ **Output:** Boxes appear perfectly centered.

💡 Common in login forms, modals, hero sections, etc.

🌟 **LEVEL 6: Column Alignment Challenge**

🎯 **Goal:** Vertically stacked boxes with space between.

```
.container {
  display: flex;
  flex-direction: column;
  justify-content: space-around;
  align-items: center;
  height: 300px;
}
```

✅ **Output:**



Box 1

(space)

Box 2

(space)

Box 3

LEVEL 7: Mini LeetCode Challenge

 **Goal:** Arrange 4 boxes evenly spaced in one row, vertically centered.

 **Solution:**

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: center;  
}
```

 **Output:**

 —  —  —  (centered vertically)

LEVEL 8: Real-World Simulation (Navigation Bar)

 **HTML**

```
<nav class="navbar">  
  <div class="logo">MySite</div>  
  <ul class="menu">  
    <li>Home</li>  
    <li>About</li>  
    <li>Courses</li>  
    <li>Contact</li>  
  </ul>  
</nav>
```

 **CSS**

```
.navbar {
```

```

display: flex;
justify-content: space-between;
align-items: center;
background-color: #2196f3;
color: white;
padding: 10px 20px;
}
.menu {
display: flex;
gap: 20px;
list-style: none;
}

```

✅ **Output:**

Logo on left → Menu on right → Perfect alignment using Flexbox.

🧩 **LEVEL 9: Debug Challenge**

❌ **Wrong:**

```

.container {
display: flex;
justify-content: bottom; /* ❌ Invalid value */
}

```

✅ **Correct:**

```

.container {
display: flex;
align-items: flex-end; /* ✅ for bottom alignment */
}

```

💡 justify-content handles **horizontal**,
align-items handles **vertical**.

🎯 **FINAL LEVEL: “Flexbox Boss Challenge”**

Goal: Create layout like:

| Header |

```
|-----|
| Sidebar | Main |
| Footer   |
```

✅ HTML

```
<div class="layout">
  <header>Header</header>
  <div class="middle">
    <aside>Sidebar</aside>
    <main>Main</main>
  </div>
  <footer>Footer</footer>
</div>
```

✅ CSS

```
.layout {
  display: flex;
  flex-direction: column;
  height: 100vh;
}

.middle {
  display: flex;
  flex: 1;
}

header, footer {
  background: #2196f3;
  color: white;
  text-align: center;
  padding: 10px;
}

aside {
  background: lightgray;
  width: 30%;
  padding: 10px;
```

```
}  
main {  
  background: #e3f2fd;  
  flex: 1;  
  padding: 10px;  
}
```

✅ Output:

Header at top → Sidebar + Main in middle → Footer at bottom.

Index.html

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Flexbox Layout Example</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
  
<body>  
  
  <!-- Whole page wrapper -->  
  <div class="layout">  
  
    <!-- Header Section -->  
    <header>Header</header>  
  
    <!-- Middle section that contains Sidebar and Main content -->  
    <div class="middle">  
      <aside>Sidebar</aside>  
      <main>Main</main>  
    </div>  
  
    <!-- Footer Section -->
```

```
<footer>Footer</footer>
```

```
</div>
```

```
</body>
```

```
</html>
```

Style.css

```
/* Main layout container */
```

```
.layout {  
  display: flex;           /* enables flexbox */  
  flex-direction: column;  /* stacks header, middle, footer vertically */  
  height: 100vh;          /* full viewport height */  
}
```

```
/* Middle section → Sidebar + Main */
```

```
.middle {  
  display: flex;           /* enables side-by-side layout */  
  flex: 1;                /* takes remaining space between header & footer */  
}
```

```
/* Header and Footer styling */
```

```
header, footer {  
  background: #2196f3;     /* blue background */  
  color: white;            /* white text */  
  text-align: center;      /* center text horizontally */  
  padding: 10px;          /* inner spacing */  
}
```

```
/* Sidebar styling */
```

```
aside {  
  background: lightgray;   /* light gray background */  
  width: 30%;              /* takes 30% of middle width */  
}
```

```
padding: 10px;      /* inner spacing */
}

/* Main content styling */
main {
  background: #e3f2fd;    /* light blue background */
  flex: 1;               /* take remaining width (70%) */
  padding: 10px;
}
```

FLEXBOX FORMULA SHEET (Summary)

Property	Axis	Common Values	Meaning
display: flex	—	—	Enables Flexbox
flex-direction	Main axis	row, column, row-reverse, column-reverse	Controls direction
justify-content	Main axis	flex-start, center, flex-end, space-*	Horizontal alignment
align-items	Cross axis	flex-start, center, flex-end, stretch	Vertical alignment
gap	—	px/%/em	Space between items
flex-wrap	—	wrap, nowrap	Allows items to wrap on multiple lines

What is flex?

👉 flex is a **short-hand (shortcut)** property in CSS used inside a **Flexbox container** (for child items).

It is written like this:

```
flex: grow shrink basis;
```

It is a combination of 3 properties:

Part Full Property Meaning

1 flex-grow How much an item will **grow** (expand) compared to others

Part Full Property Meaning

- 2** flex-shrink How much an item will **shrink** when space is tight
 - 3** flex-basis The **initial size** of the item before growing or shrinking
-

💡 Example 1: flex: 1

```
.box {  
  flex: 1;  
}
```

This is actually short for:

```
.box {  
  flex-grow: 1;  
  flex-shrink: 1;  
  flex-basis: 0;  
}
```

🔍 Meaning:

- Each box will **grow equally** to fill all remaining space.
 - All boxes with the same flex: 1 will have **equal width** (if direction = row).
-

🧠 Example Visualization

```
<div class="container">  
  <div class="box one">1</div>  
  <div class="box two">2</div>  
  <div class="box three">3</div>  
</div>  
  
.container {  
  display: flex;  
  height: 100px;  
}  
  
.box {  
  flex: 1;
```



```
color: white;

text-align: center;

line-height: 100px;
}

.one { background: tomato; }
.two { background: teal; }
.three { background: orange; }
```

✅ Output:

All 3 boxes are equal in width (1/3 each of total container width).

💡 Example 2: flex: 1 vs flex: 2

```
.one { flex: 1; }
.two { flex: 2; }
.three { flex: 1; }
```

🔍 Meaning:

- .two will grow **twice as much** as .one or .three.
- Total “growth units” = $1 + 2 + 1 = 4$.
- So, container space is divided into 4 parts:
 - Box 1 → 1 part
 - Box 2 → 2 parts
 - Box 3 → 1 part

✅ Result:

If container width = 400px →

- Box 1 = 100px
 - Box 2 = 200px
 - Box 3 = 100px
-

💡 Example 3: flex-basis in action

```
.box1 { flex: 1 1 100px; }
.box2 { flex: 2 1 100px; }
```

- Both boxes start with **100px** width (flex-basis: 100px).
- Box 2 will grow twice as much as Box 1 if extra space is available.

Summary Table

Property	Meaning	Example
flex-grow	Defines how much element expands	0 = no grow, 1 = grow, 2 = double grow
flex-shrink	Defines how much element shrinks when space is small	0 = no shrink, 1 = shrink normally
flex-basis	Starting width/height before grow/shrink	e.g., 100px, 50%, or auto
flex	Shortcut of all 3	e.g., flex: 1 → grow=1, shrink=1, basis=0

Real-Life Example (inside a layout)

```
aside {  
  width: 30%;  
}  
  
main {  
  flex: 1;  
}
```

Meaning:

- Sidebar (aside) has a fixed width.
 - Main content (main) uses remaining space (because of flex: 1).
-

Easy Analogy:

Think of Flexbox like **dividing pizza**  among friends:

- flex: 1 → everyone gets **equal slices**.
- flex: 2 → that person gets **double the slice**.
- flex: 3 → triple slice compared to one who has flex: 1.